

EE24101 Basic Electronics

Module 5: Boolean Algebra and Logic Gates

Dr. Jyoti Sharma

BIT Mesra, Jaipur Campus

November 27, 2025



Overview

- 1 Boolean Algebra
- 2 Boolean Operators
- 3 Truth table of different logic gates (AND, OR, NOT, NAND, NOR, EXOR, EXNOR)
- 4 Application of diode for design of logic gates
- 5 Realisation of logic gates using Universal gates
- 6 Adders
- 7 Subtractor

Boolean Algebra

- Boolean algebra deals with binary variables: 0 (False) and 1 (True).
- Developed by George Boole in the 19th century.
- It is described as an algebra of logic or an algebra of two values i.e True or False.
- The term logic means a statement having binary decisions i.e True/Yes or False/No.
- Used to simplify and analyze digital circuits.
- It is used to perform the logical operations in digital computer.
- In digital computer True represent by '1' (high volt) and False represent by '0' (low volt)
- Logical operations are performed by logical operators. The fundamental logical operators are:
 1. AND (conjunction)
 2. OR (disjunction)
 3. NOT (negation/complement)

AND operator

- It performs logical multiplication and denoted by (.) dot.

X	Y	X.Y
0	0	0
0	1	0
1	0	0
1	1	1

OR operator

- It performs logical addition and denoted by (+) plus.

X	Y	X.Y
0	0	0
0	1	1
1	0	1
1	1	1

NOT operator

- It performs logical negation and denoted by (-) bar. It operates on single variable.

X	$\overline{X}$
0	1
1	0

Truth Table

- Truth table is a table that contains all possible values of logical variables/statements in a Boolean expression.
- No. of possible combination = 2^n , where n =number of variables used in a Boolean expression.
- The truth table for $XY + Z$ is as follows:

X	Y	Z	XY	XY+Z
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	0	1

- A gate is an digital circuit which operates on one or more signals and produce single output.
- Gates are digital circuits because the input and output signals are denoted by either 1 (high voltage) or 0 (low voltage).
- There are three basic gates:
 1. AND gate
 2. OR gate
 3. NOT gate

AND gate

- The AND gate is an electronic circuit that gives a high output (1) only if all its inputs are high.
- AND gate takes two or more input signals and produce only one output signal.



Figure 1: AND gate

Input A	Input B	Output AB
0	0	0
0	1	0
1	0	0
1	1	1

OR gate

- The OR gate is an electronic circuit that gives a high output (1) if one or more of its inputs are high.
- OR gate also takes two or more input signals and produce only one output signal.

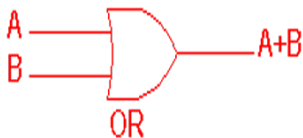


Figure 2: OR gate

Input A	Input B	Output AB
0	0	0
0	1	1
1	0	1
1	1	1

NOT gate

- The NOT gate is an electronic circuit that gives a high output (1) if its input is low.
- NOT gate takes only one input signal and produce only one output signal.
- The output of NOT gate is complement of its input. It is also called inverter.

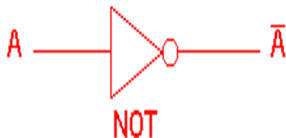


Figure 3: NOT gate

Input	Output
A	$\bar{A}$
0	1
1	0

NAND Gate

- Known as a “universal” gate because any digital circuit can be implemented with NAND gates alone.

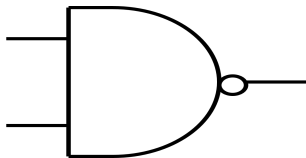


Figure 4: NAND gate

Input A	Input B	Output Z
0	0	1
0	1	1
1	0	1
1	1	0

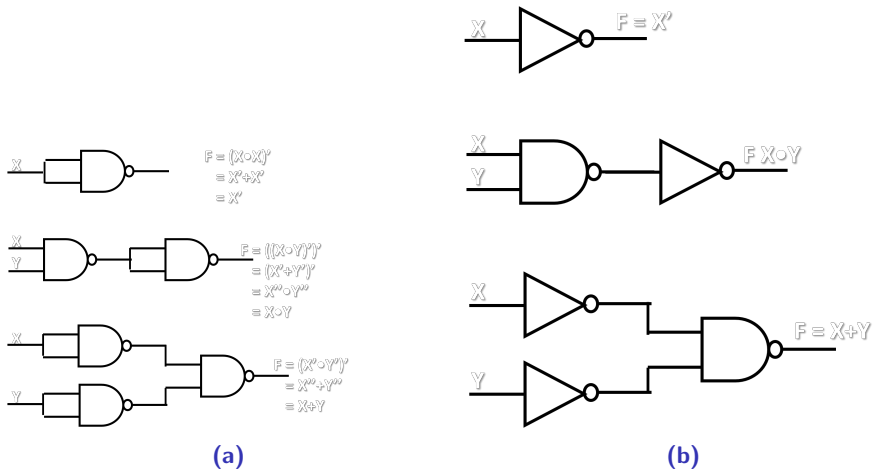


Figure 5: NOT, AND and OR gates using NAND gate

NOR Gate

- Known as a “universal” gate because any digital circuit can be implemented with NOR gates alone.

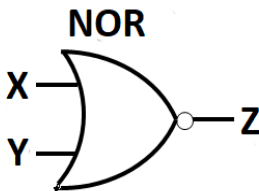


Figure 6: NOR gate

Input A	Input B	Output Z
0	0	1
0	1	0
1	0	0
1	1	0

AND, OR, and NOT gate implementation using NOR gate

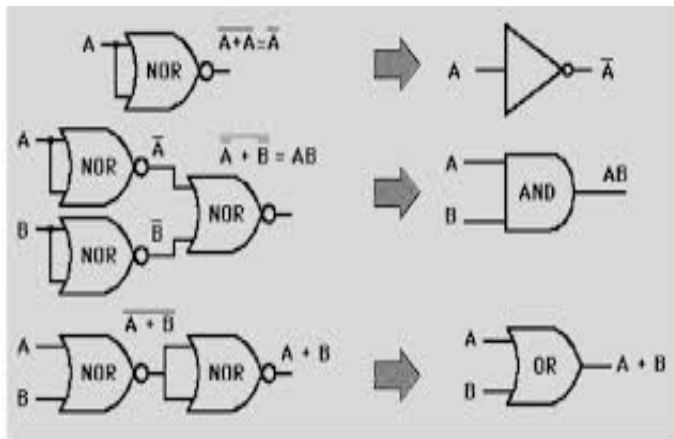


Figure 7

Exclusive-OR Gate

- An exclusive OR (XOR) gate is a digital logic circuit with two inputs and one output. It produces a high output (1) only when the inputs are different, and a low output (0) when the inputs are the same.
- This makes it useful for applications like arithmetic circuits, error detection, and as an inequality detector.



Figure 8: EXOR gate

Input A	Input B	Output Z
0	0	0
0	1	1
1	0	1
1	1	0

Exclusive NOR Gate

- An exclusive NOR (XNOR) gate is a digital logic gate that produces a high output (1) only when both inputs are the same (0 and 0, or 1 and 1). It is the logical opposite of an exclusive OR (XOR) gate and is sometimes called the "gate of equivalence" because it outputs a 1 when the inputs are equal. The XNOR gate is formed by combining an XOR gate with a NOT gate.



Figure 9: EXNOR gate

Input A	Input B	Output Z
0	0	1
0	1	0
1	0	0
1	1	1

Boolean Laws and Theorems

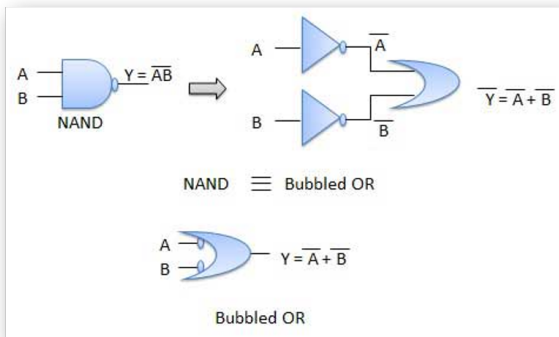
- Identity Law: $A + 0 = A$, $A \bullet 1 = A$
- Null Law: $A + 1 = 1$, $A \bullet 0 = 0$
- Idempotent (Identity) Law: $A + A = A$, $A \bullet A = A$
- Complement Law: $A + A' = 1$, $A \bullet A' = 0$
- Commutative law: $A + B = B + A$; $AB = BA$
- Associative law: $(A + B) + C = A + (B + C)$; $(AB)C = A(BC)$
- Distributive Law: $A(B + C) = AB + AC$;
 $A + (BC) = (A + B)(A + C)$ $A + A'B = A + B$
- De Morgan's Theorem:
 $(A \cdot B)' = A' + B'$
 $(A + B)' = A' \cdot B'$
- Involution law: $A'' = A$
- Absorption law: $A + (A \cdot B) = A$; $A(A + B) = A$

De Morgan's Theorem 1

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

NAND = Bubbled OR

Figure 10



A	B	$\overline{AB}$	$\overline{A}$	$\overline{B}$	$\overline{A} + \overline{B}$
0	0	1	1	1	1
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	0	0	0

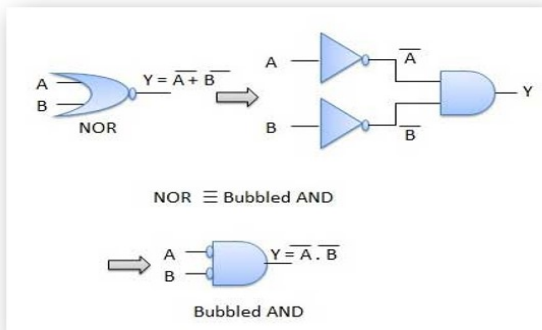
Figure 12

De Morgan's Theorem 2

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

NOR = Bubbled AND

Figure 13



A	B	$\overline{A+B}$	$\overline{A}$	$\overline{B}$	$\overline{A} \cdot \overline{B}$
0	0	1	1	1	1
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	0

Figure 15

Application of Diode for design of logic gates

- OR Gate

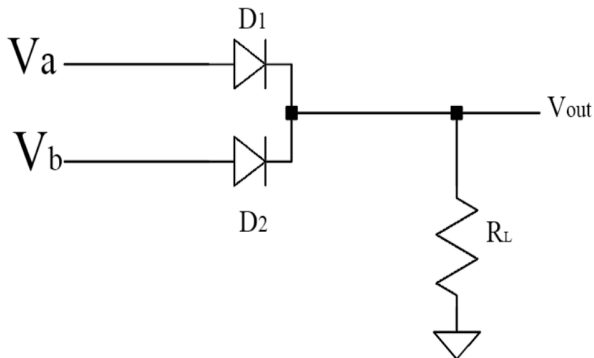


Figure 16

- If all inputs are in LOW, both the diode becomes in reverse biased hence acts as an open switch. Hence the output voltage is LOW.
- If A is HIGH and B is LOW, the diode D1 becomes in forward biased hence act as the closed switch. (Neglecting diode forward resistance and voltage drop across the diode) Hence the output is HIGH.
- Also, If A is LOW and B is HIGH. Diode D2 becomes in forward biased and act as an open switch. Hence the output is HIGH.
- If both the input is in HIGH then the output is equal to the more positive value of the input.
- Hence OR function has been implemented.

- AND gate:

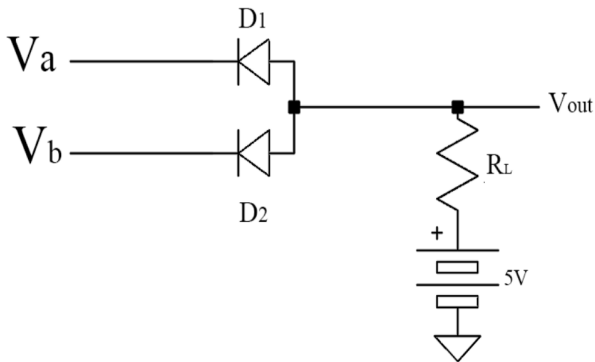


Figure 17

- If A is HIGH and b is LOW, the diode D1 becomes in reversed biased hence act as an open switch. Also, diode D2 becomes in forward biased hence act as the closed switch. Hence the output is LOW.
- If A is LOW and B is HIGH, the diode D1 becomes in forward biased hence act as the closed switch. Also, diode D2 becomes in reversed biased hence act as an open switch. The output is LOW.
- If all inputs are LOW then all diode becomes in forward biased and act as an open switch. Hence the output is LOW.
- When all inputs are HIGH then all diodes becomes in the reversed biased hence act as an open switch. Hence the output is HIGH.

- NOT gate:

- If the input is LOW, the transistor act as an open switch. Hence the output is HIGH.
- If the input is HIGH, the transistor act as the closed switch. Hence the output is LOW

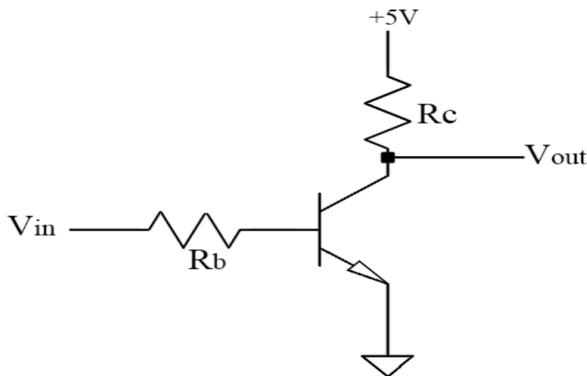


Figure 18

HALF ADDER

- An adder is a digital circuit that performs addition of numbers.
- Adders are classified into two types .
- One is Half Adder, and another one is known as Full Adder.
- The half adder adds two binary digits and produces two outputs as sum and carry.
- It consists of one XOR logic gate producing “SUM” and one AND gate producing “CARRY” as outputs.

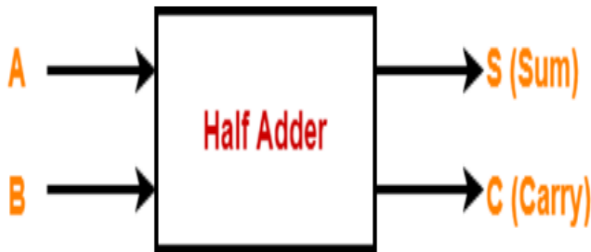


Figure 19

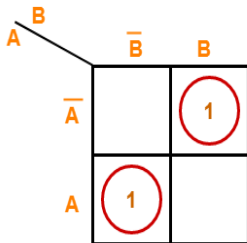
Truth table

INPUTS		OUTPUTS	
A	B	S(Sum)	C(Carry)
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

Figure 20

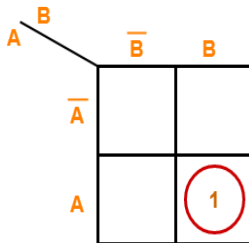
K - Maps

For S:



$$S = A \oplus B$$

For C:



$$C = A \cdot B$$

K Maps

Figure 21

logic diagram

- There are two inputs and two outputs in a Half Adder. Inputs are named as A and B, and the outputs are named as Sum (S) and Carry (C).

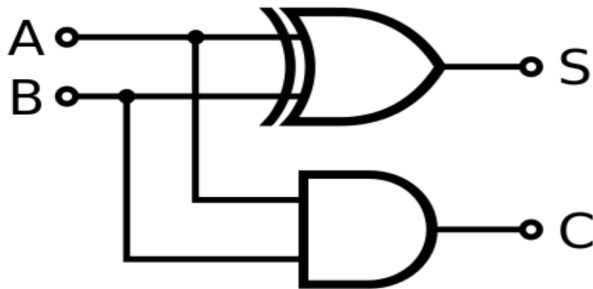


Figure 22

Limitation of Half Adder

- The major drawback of HALF ADDER circuit is that it can only add two inputs and if there is any carry it is neglected.
- Thus, the process is incomplete. To overcome this drawback Full Adder is designed.

Full Adder

- A full adder is a combinational circuit that performs an addition operation on three binary digits.
- It consists of three inputs and two outputs.
- The first two inputs are A and B and the third input is an input carry designated as C_{in} .
- The output carry is designated as C_{out} and the normal output is designated as S.

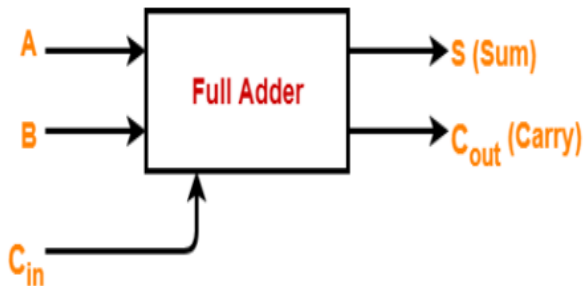


Figure 23

Truth table

INPUTS			OUTPUTS	
A	B	Cin	S(sum)	Cout (carry)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Figure 24

K - Maps

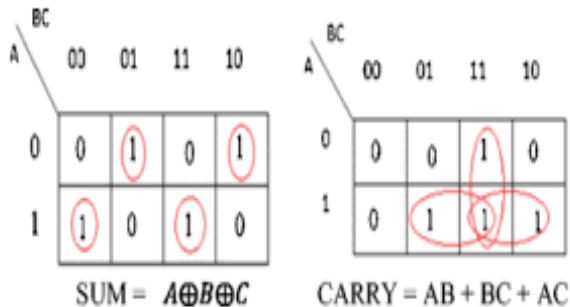


Figure 25

- A full adder circuit can be implemented using two half adders and one OR gate.
- The first half adder will be used to add A and B to produce a partial Sum and the second half adder can be used to add C_{in} to the Sum produced by the first half adder to get the final S output.
- If any of the half adder produces a carry, there will be an output carry. So, C_{out} will be an OR function of the half adder Carry outputs.
- Since a full adder has the previous carry as an input, it can be used to add multi-digit binary numbers.
- The multi-digit binary numbers are added by a cascading connection of multiple full adders.
- The number of full adders in cascade connection is equal to the number of digits to be added.

Full Adder Circuit using Half Adders and OR gate

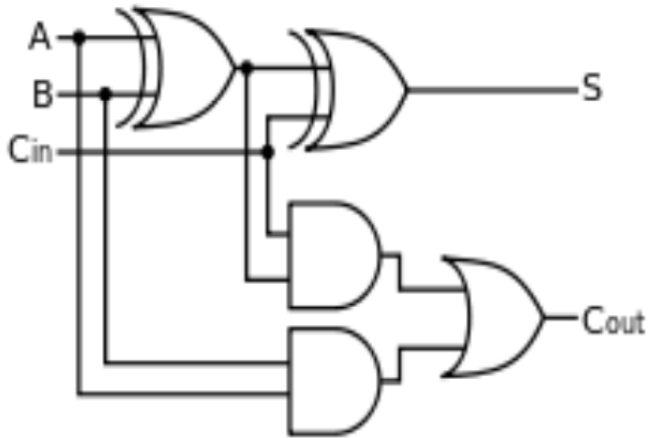


Figure 26

HALF SUBTRACTOR

- Half Subtractor is a combinational logic circuit.
- It is used for the purpose of subtracting two single bit numbers.
- It contains 2 inputs and 2 outputs (difference and borrow).

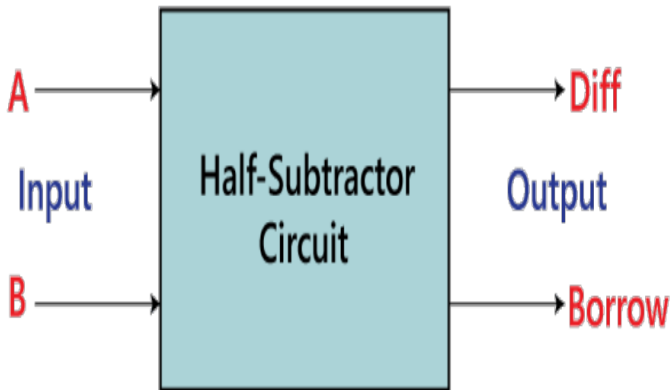


Figure 27

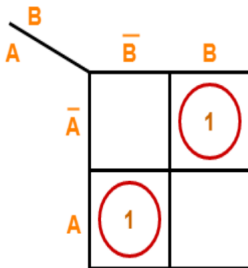
Truth table

INPUTS		OUTPUTS	
A	B	D (Difference)	B (Borrow)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Figure 28

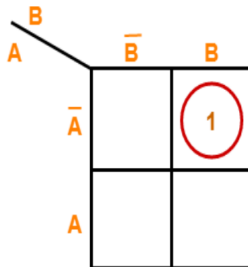
K - Maps

For D:



$$D = A \oplus B$$

For b:



$$b = \bar{A}.B$$

Figure 29

Logic Diagram

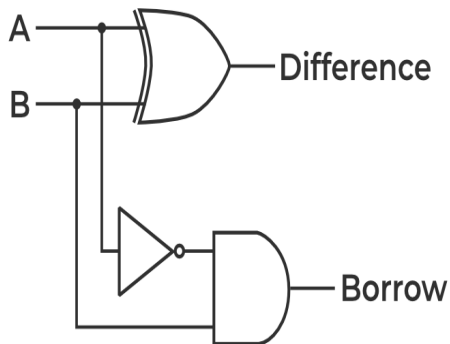


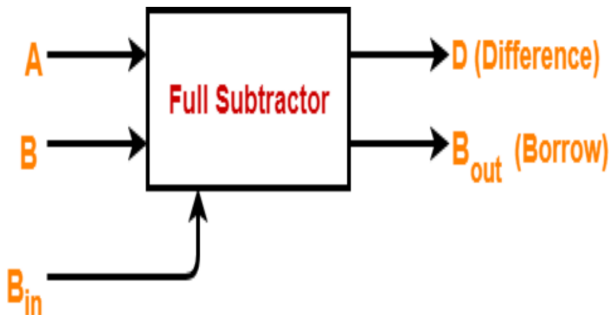
Figure 30

Limitation of Half Subtractor

- Half subtractors do not take into account “Borrow-in” from the previous circuit.
- This is a major drawback of half subtractors.
- This is because real time scenarios involve subtracting the multiple number of bits which can not be accomplished using half subtractors.
- To overcome this drawback, Full Subtractor comes into play.

Full Subtractor

- Full Subtractor is a combinational logic circuit.
- It is used for the purpose of subtracting two single bit numbers.
- It also takes into consideration borrow of the lower significant stage.
- Thus, full subtractor has the ability to perform the subtraction of three bits.
- Full subtractor contains 3 inputs and 2 outputs (Difference and Borrow) as shown:



Truth Table

INPUTS			OUTPUTS	
A	B	B_{in}	$B_{out}(\text{borrow})$	D(difference)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Figure 32

K - Maps

K-maps

For D:

A \ B _{in}	$\overline{B}\overline{B}_{in}$	$\overline{B}B_{in}$	BB_{in}	$B\overline{B}_{in}$
$\overline{A}$		1		1
A	1		1	

$$D = A \oplus B \oplus B_{in}$$

For B_{in}:

A \ B _{in}	$\overline{B}\overline{B}_{in}$	$\overline{B}B_{in}$	BB_{in}	$B\overline{B}_{in}$
$\overline{A}$		1	1	1
A			1	

$$B_{out} = \overline{A} B + (\overline{A} + B) B_{in}$$

Figure 33

Logic Diagram

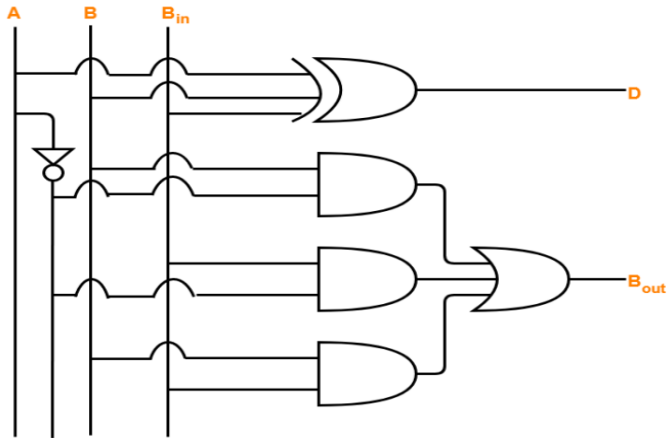


Figure 34

Minimization of Boolean Functions using K- Map

1- Two variables K-Map

Number of input variables are 2

Hence the number of squares = $2^n = 2^2 = 4$

Inputs A B		Decimal equivalent	Minterms		Output F
0	0	0	m_0	$\overline{A}\overline{B}$	
0	1	1	m_1	$\overline{A}B$	
1	0	2	m_2	$A\overline{B}$	
1	1	3	m_3	AB	

And K-Map of two variables is:

	$\overline{B}$ 0	B 1
$\overline{A}$ 0	0	1
A 1	2	3

Example: simplify the Boolean expression by using K-Map

$$F = \bar{A}B + AB$$

Solution:

Number of input variables are 2

Hence the number of squares = $2^n = 2^2 = 4$

	$\bar{B}$	B
$\bar{A}$		
0	0	1
A		
1	0	1

$$F = B$$

Figure 36

Example: simplify the Boolean expression by using K-Map

$$F(A, B) = \sum m(2, 0, 3)$$

Solution:

Number of input variables are 2

Hence the number of squares = $2^n = 2^2 = 4$

	$\bar{B}$ 0	B 1
$\bar{A}$ 0	0 1	1 0
A 1	2 1	3 1

$$F(A, B) = \bar{B} + A$$

Figure 37

Example: simplify the Boolean expression by using K-Map

$$F = \bar{A}B + A\bar{B}$$

Solution:

Number of input variables are 2

Hence the number of squares = $2^n = 2^2 = 4$

	$\bar{B}$ 0	B 1
$\bar{A}$ 0	0 1	1 1
A 1	2 0	3 0

$$F = \bar{A}$$

Figure 38

Example: simplify the Boolean expression by using K-Map

$$F(A, B) = \sum m(0, 3)$$

Solution:

Number of input variables are 2

Hence the number of squares = $2^n = 2^2 = 4$

	$\bar{B}$ 0	B 1
$\bar{A}$ 0	1 ⁰	0 ¹
A 1	0 ²	1 ³

$$F(A, B) = \sum m(0, 3) = \bar{A}\bar{B} + AB$$

Figure 39

2- Three Variables K-Map

Number of input variables are 3

Hence the number of squares = $2^n = 2^3 = 8$

The truth table is

Inputs A B C			Decimal equivalent	Minterms		Output F
0	0	0	0	m_0	$\overline{A}\overline{B}\overline{C}$	
0	0	1	1	m_1	$\overline{A}\overline{B}C$	
0	1	0	2	m_2	$\overline{A}B\overline{C}$	
0	1	1	3	m_3	$\overline{A}BC$	
1	0	0	4	m_4	$A\overline{B}\overline{C}$	
1	0	1	5	m_5	$A\overline{B}C$	
1	1	0	6	m_6	$AB\overline{C}$	
1	1	1	7	m_7	ABC	

Figure 40

	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	BC 11	$B\overline{C}$ 10
$\overline{A}$ 0	0	1	3	2
A 1	4	5	7	6

Figure 41

Example: simplify the Boolean expression by using K-Map

$$F(A, B, C) = \overline{A}\overline{B}\overline{C} + \overline{A}BC + \overline{A}B\overline{C}$$

Solution:

Number of input variables are 3

Hence the number of squares = $2^n = 2^3 = 8$

	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	BC 11	$B\overline{C}$ 10
$\overline{A}$ 0	1	0	1	1
A 1	0	0	0	0

$$F(A, B, C) = \overline{A}\overline{C} + \overline{A}B$$

Figure 42

Example: simplify the Boolean expression by using K-Map

$$F(A, B, C) = \sum m(0, 3, 7, 6)$$

Solution:

Number of input variables are 3

Hence the number of squares = $2^n = 2^3 = 8$

	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	BC 11	$B\overline{C}$ 10
$\overline{A}$ 0	0 1	1 0	3 1	2 0
A 1	4 0	5 0	7 1	6 1

$$F(A, B, C) = \overline{A}\overline{B}\overline{C} + BC + AB$$

Figure 43

3- Four Variables K-map

Number of input variables are 4

Hence the number of squares = $2^n = 2^4 = 16$

The truth table is

Inputs				Decimal equivalent	Minterms		Output F
A	B	C	D				
0	0	0	0	0	m_0	$\overline{A}\overline{B}\overline{C}\overline{D}$	
0	0	0	1	1	m_1	$\overline{A}\overline{B}\overline{C}D$	
0	0	1	0	2	m_2	$\overline{A}\overline{B}C\overline{D}$	
0	0	1	1	3	m_3	$\overline{A}\overline{B}CD$	
0	1	0	0	4	m_4	$\overline{A}B\overline{C}\overline{D}$	
0	1	0	1	5	m_5	$\overline{A}B\overline{C}D$	
0	1	1	0	6	m_6	$\overline{A}BC\overline{D}$	
0	1	1	1	7	m_7	$\overline{A}BCD$	
1	0	0	0	8	m_8	$A\overline{B}\overline{C}\overline{D}$	
1	0	0	1	9	m_9	$A\overline{B}\overline{C}D$	
1	0	1	0	10	m_{10}	$A\overline{B}C\overline{D}$	
1	0	1	1	11	m_{11}	$A\overline{B}CD$	
1	1	0	0	12	m_{12}	$AB\overline{C}\overline{D}$	
1	1	0	1	13	m_{13}	$AB\overline{C}D$	
1	1	1	0	14	m_{14}	$ABCD\overline{D}$	
1	1	1	1	15	m_{15}	$ABCD$	

Figure 44

And the K-Map of four variables is:

	$\overline{C}\overline{D}$ 00	$\overline{C}D$ 01	CD 11	$C\overline{D}$ 10
$\overline{A}\overline{B}$ 00	0	1	3	2
$\overline{A}B$ 01	4	5	7	6
AB 11	12	13	15	14
$A\overline{B}$ 10	8	9	11	10

Figure 45

Example: simplify the Boolean expression by using K-Map

$$F(A, B, C, D) = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + A\bar{B}\bar{C}\bar{D} + A\bar{B}C\bar{D} + \bar{A}BCD + A\bar{B}CD$$

Solution: Number of input variables are 4

Hence the number of squares = $2^n = 2^4 = 16$

	$\bar{C}\bar{D}$ 00	$\bar{C}D$ 01	CD 11	$C\bar{D}$ 10
$\bar{A}\bar{B}$ 00	0 1	1 0	3 0	2 1
$\bar{A}B$ 01	4 0	5 0	7 1	6 0
AB 11	12 0	13 1	15 0	14 0
$A\bar{B}$ 10	8 1	9 0	11 0	10 1

$$F(A, B, C, D) = \bar{B}\bar{D} + \bar{A}BCD + A\bar{B}CD$$

Figure 46

Example: simplify the Boolean expression by using K-Map

$$F(A,B,C,D) = \sum m(0,2,4,6,12,14,15,8,10)$$

Solution: Number of input variables are 4

Hence the number of squares = $2^n = 2^4 = 16$

	$\overline{C}\overline{D}$ 00	$\overline{C}D$ 01	CD 11	$C\overline{D}$ 10
$\overline{A}\overline{B}$ 00	0 1	1 0	3 0	2 1
$\overline{A}B$ 01	4 1	5 0	7 0	6 1
AB 11	12 1	13 0	15 1	14 1
$A\overline{B}$ 10	8 1	9 0	11 0	10 1

$$F(A,B,C,D) = \overline{D} + ABC$$

Figure 47

K-Map with Don't Care Conditions

- In certain cases some of the minterms may never occur or it may not matter what happens if they do
- In such cases we fill in the Karnaugh map with an X that meaning don't care
- X can be 0 or 1 - whatever helps best with the minimization

Example: simplify the Boolean expression by using K-Map

$$F(A, B, C, D) = \sum m(3, 7, 9, 11) + \sum d(1, 5, 12, 14)$$

Solution: Number of input variables are 4

Hence the number of squares = $2^n = 2^4 = 16$

	$\overline{C}\overline{D}$ 00	$\overline{C}D$ 01	CD 11	$C\overline{D}$ 10
$\overline{A}\overline{B}$ 00	0 ⁰ 0	X ¹ 1	1 ³ 1	0 ² 0
$\overline{A}B$ 01	0 ⁴ 0	X ⁵ 1	1 ⁷ 1	0 ⁶ 0
AB 11	X ¹² 1	0 ¹³ 0	0 ¹⁵ 0	X ¹⁴ 1
$A\overline{B}$ 10	0 ⁸ 0	1 ⁹ 1	1 ¹¹ 1	0 ¹⁰ 0

$$F(A, B, C, D) = \overline{A}D + \overline{B}D$$

Figure 48

- Prove the theorem of Absorption in Boolean Algebra. Express the following function in a sum of minterms and a product of maxterms.

$$F(w, x, y, z) = y'z + wxy' + wxz' + w'x'z \quad (1)$$

- Construct a full-subtractor circuit with two half subtractors and an OR gate.
- Simplify the Boolean expression $F = (W + W'X + Y'Z + Y)$ and prove that $F = (W + X + Y + Z)$. Also realize it using NOR gates.
- What is the difference between combinational and sequential circuits? Write the truth table of a full-subtractor and construct its logic circuit.

END OF MODULE 5